Studio Nx

백엔드 156 — issued_before_password_change()

2026-10-14

백엔드 156 — `issued_before_password_change()` (이전 로그인 무효 판단)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 일곱 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	155 `reset_password()`
시험	`tests/test_reset_token_hash.py`의 14개 (이 파일 전체 31개)
다음	이 판단을 `get_current_user`(액세스 토큰)와 `/auth/refresh`(리프레시 토큰)에 연결

1. 하는 일

로그인 토큰이 비밀번호를 재설정하기 **전에** 발급된 것이면 `True`(= 무효)를 돌려주는 **판단 함수**입니다. 비밀번호를 잊어서 재설정한다면 그 계정에 이미 들어와 있던 다른 사람(또는 도난당한 토큰)을 내보내는 것이 목적입니다.

입력	결과
재설정한 적이 없음(`password_changed_at`이 `None`)	항상 `False` (기존 로그인은 그대로)
`iat`가 재설정 시각의 초보다 **앞**	`True` (무효)
`iat`가 재설정 시각의 초와 **같거나 뒤**	`False` (유효)
재설정했는데 `iat`가 없거나 숫자가 아님	`True` (확인할 수 없으면 거부)

2. 코드

```
def issued_before_password_change(issued_at: object, password_changed_at: datetime | None) -> bool:
    if password_changed_at is None:
        return False
    if isinstance(issued_at, bool) or not isinstance(issued_at, int | float):
        return True
    changed = calendar.timegm(password_changed_at.utctimetuple())
    return issued_at < changed
```

3. 왜 이렇게

선택	이유
초 단위로 비교	JWT의 `iat`는 초 단위이고 `password_changed_at`은 밀리초까지 있습니다. 그대로 비교하면 재설정 직후 **같은 초**에 새로 로그인한 사람의 토큰이 "이전 것"으로 잘못 거부됩니다. 그래서 `password_changed_at`의 초 미만을 버리고 `iat < 그 초`일 때만 무효로 봅니다
같은 초의 토큰은 **유효**	반대쪽 위험(재설정과 같은 초에 앞서 발급된 옛 토큰이 살아남음)은 현실적으로 없습니다. 재설정은 메일 링크를 거쳐서, 같은 초에 앞선 로그인이 있을 수 없습니다

선택	이유
`calendar.timegm(….utctimetuple())`	DB의 시각은 UTC이고 시간대 정보가 없습니다. `datetime.timestamp()`는 시간대 정보가 없는 값을 **서버의 로컬 시간**으로 읽어서 서버 시간대에 따라 결과가 달라집니다. `timegm`은 항상 UTC로 읽습니다
이상한 `iat`는 무효	재설정한 계정에서 `iat`가 없거나 문자열이거나 `True`인 토큰은 우리가 만든 토큰이 아닙니다. 확인할 수 없으면 거부하는 쪽이 안전합니다. `bool`은 `int`의 하위 형이라 따로 걸러야 합니다
순수 함수로 분리	DB나 토큰 해독 없이 시각 비교만 해서 경계 시험을 정확히 할 수 있습니다. 실제 연결은 다음 함수가 합니다

4. 시험 (14개 추가, 파일 전체 31개 통과)

확인	결과
재설정한 적이 없으면 항상 유효(`iat`가 이상해도)	✓
재설정 이전 발급은 무효	✓
재설정 이후 발급은 유효	✓
같은 초 발급은 유효(밀리초 .000, .500, .999 모두)	✓
초 경계 바로 앞은 무효, 바로 위는 유효	✓
소수 `iat`는 값으로 비교	✓
`None`, 문자열, `True`, `False`, 리스트, 딕셔너리, `bytes`인 `iat`는 무효 (7개)	✓
UTC로 읽음(서버 시간대와 무관)	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`<`를 `<=`로	2개 실패
초 미만을 버리지 않고 밀리초까지 비교	1개 실패 (**같은 초의 새 로그인이 거부되는 오류**를 잡음)
`None`일 때의 빠른 반환 제거	1개 실패
이상한 `iat`를 유효로	7개 실패
`timegm` 대신 로컬 시간(`timestamp()`)	4개 실패
`bool` 허용	**실패 없음**

마지막은 `True`(=1)를 `iat`로 받아도 `1 < 재설정 시각`이라 어차피 `True`(무효)가 되어 결과가 같기 때문입니다. `bool` 검사는 `iat=True`가 "유효한 시각"처럼 취급되지 않게 하는 방어이고, 동작을 가르는 경계가 없어 시험으로 잡을 수 없는 중복 방어입니다.

전체 pytest 754개 통과, `ruff` 통과.

5. 다음 함수에 남기는 일

이 판단을 쓰려면 토큰에서 `iat`를 꺼내야 하는데, 지금 `decode_access_token`과 `decode_refresh_token`은 `user_id`만 돌려줍니다. 다음 함수에서 다음 중 하나로 연결해야 합니다.

1. `iat`를 함께 돌려주는 새 해독 함수를 더하고(기존 함수와 시험은 그대로), `get_current_user`와 `/auth/refresh`가 그것을 씁니다.
2. 기존 함수의 반환값을 `(user_id, iat)`로 바꿉니다. 호출하는 곳과 시험이 많이 바뀝니다.

1번을 권합니다. 기존 시험과 호출부를 건드리지 않기 때문입니다.